

Contents

1	phd_configuration	1
1.1	Repository role	1
1.2	Files	1
1.3	Use as a submodule	1
1.4	Imports	2
1.5	Runtime configuration	2
1.6	Run-folder workflow	2
1.7	Stage tracking helpers	3
1.8	Geometry helpers	3
1.9	Logging helpers	4
1.10	Configuration keys	4
1.11	Development notes	4

1 phd_configuration

Shared configuration module for the PhD seismic processing projects.

This repository is intended to be used as a Git submodule named `phd_configuration` inside the other project repositories, for example `phd_data_acquisition`, `phd_preprocessing`, and `phd_visualization`. Those projects import the same configuration and helper functions instead of keeping separate local copies.

1.1 Repository role

`phd_configuration` provides:

- A central `config.json` with data paths, acquisition thresholds, preprocessing parameters, and `STUDY_AREA` geometry.
- A runtime config loader in `config.py`.
- Shared utility functions in `utils.py` for JSON I/O, QC/run folders, stage tracking, coordinate conversion, ZNE-to-BOX rotation, and logging.

The module is deliberately small because it is a common dependency. Changes in this repository can affect all projects that include it as a submodule.

1.2 Files

File	Purpose
<code>config.json</code>	Base configuration used by all projects.
<code>config.py</code>	Loads the base config or the active run config.
<code>utils.py</code>	Shared helpers used by acquisition, preprocessing, and visualization scripts.
<code>README.md</code>	Documentation for this shared module.

1.3 Use as a submodule

Add this repository inside another project with:

```
git submodule add https://github.com/iarvanitis69/phd_configuration.git phd_configuration
git submodule update --init --recursive
```

After cloning a project that already contains this submodule, initialize it with:

```
git submodule update --init --recursive
```

Update the submodule in a parent project with:

```
cd phd_configuration
git pull
cd ..
git add phd_configuration
git commit -m "Update phd_configuration submodule"
```

1.4 Imports

Typical usage from a parent repository:

```
from phd_configuration import config
from phd_configuration.utils import get_qc_folder_path, geo_to_cartesian_wrapper

logs_dir = config.get("LOGS_DIR")
qc_dir = get_qc_folder_path()
x, y, z = geo_to_cartesian_wrapper(lat, lon, depth_km)
```

The parent repository must be run from a location where Python can import the `phd_configuration` directory. This is normally true when scripts are executed from the parent repository root and the submodule path is `./phd_configuration`.

1.5 Runtime configuration

`config.py` exposes the following public functions:

Function	Description
<code>config.get(key, default=None)</code>	Read a config value from the active config.
<code>config.all()</code>	Return the active config dictionary.
<code>config.base_all()</code>	Return the base config.json dictionary from this repository.
<code>config.source_path()</code>	Return the path of the config file currently being used.
<code>config.activate(path)</code>	Manually switch the active config to another JSON file.

By default, the active config is `phd_configuration/config.json`.

If the parent repository root contains `session_info.txt` and it stores a run folder name, `config.py` checks:

```
<LOGS_DIR>/<folder name>/config.json
```

If that file exists, it becomes the active config. This lets later pipeline stages use the exact config snapshot that was written when the run started.

1.6 Run-folder workflow

The run-folder helpers in `utils.py` coordinate pipeline stages through the parent repository file:

```
session_info.txt
```

Important helpers:

Function	Description
<code>start_new_qc_folder()</code>	Creates a timestamped folder under LOGS_DIR, writes a run config.json, activates it, and stores the folder name in session_info.txt.
<code>start_or_resume_qc_folder()</code>	Resumes the folder in session_info.txt, or creates a new one if the file is empty.
<code>get_qc_folder_name()</code>	Returns the active run folder name, or raises an error if no run is active.
<code>get_qc_folder_path()</code>	Returns and creates the active run folder path under LOGS_DIR.
<code>resolve_stage_output_dir(output_dir=None)</code>	Resolves a stage output path and verifies it matches the active run.
<code>latest_qc_folder_name()</code>	Finds the latest run folder that contains a config.json.
<code>clear_session_info()</code>	Clears the active run and reactivates the base config.

The intended pipeline behavior is:

1. The first stage starts or resumes a run.
2. Every later stage writes into the active run folder.
3. The final stage clears session_info.txt after successful completion.

1.7 Stage tracking helpers

utils.py also includes small helpers for JSON tracking files:

Function	Description
<code>load_json(path)</code>	Read JSON and return {} if the file is missing or invalid.
<code>save_json(path, data)</code>	Write formatted UTF-8 JSON.
<code>get_stage_tracking_path(output_dir, stage_name)</code>	Build <output_dir>/<stage_name>.json.
<code>get_tracking_stage(data, stage_name)</code>	Extract a stage dictionary from flat or nested tracking data.
<code>stage_tracking_complete(output_dir, stage_name)</code>	Check whether a stage tracking file reports a complete stage.

1.8 Geometry helpers

The coordinate helpers use the STUDY_AREA section of config.json.

Important conventions:

- Geographic coordinates are (latitude, longitude, depth_km).
- depth_km is positive downward.
- Local BOX/STUDY_AREA z increases upward.
- The STUDY_AREA origin is bottom_SW.
- The local X, Y, and Z axes are defined from the STUDY_AREA reference corners.

Common helpers:

Function	Description
<code>geo_to_cartesian_wrapper(lat, lon, depth)</code>	Convert geographic coordinates to local Cartesian coordinates using the configured STUDY_AREA origin.
<code>compute_distance_and_angles_geo(...)</code>	Compute distance, azimuth, and polar angle between two geographic points.
<code>build_box_basis_from_config()</code>	Build ECEF-to-BOX and BOX-to-ECEF rotation matrices from <code>config.json</code> .
<code>rotate_vector_zne_to_box(...)</code>	Rotate one ZNE vector into BOX/STUDY_AREA coordinates.
<code>rotate_vector_series_zne_to_box(...)</code>	Rotate vector arrays from ZNE into BOX/STUDY_AREA coordinates.
<code>transform_traces_zne_to_box(...)</code>	Convert ObsPy Z/N/E traces into X/Y/Z BOX traces.

1.9 Logging helpers

Use `tee_stdout(logfile_path)` to mirror `print()` output both to the terminal and to a log file:

```
from phd_configuration.utils import tee_stdout
```

```
with tee_stdout("/path/to/stage.log"):
    print("This goes to stdout and the log file")
```

1.10 Configuration keys

The base `config.json` currently contains:

- Data paths: `BASE_DIR`, `LOGS_DIR`
- Acquisition settings: `min_magnitude_acquisition`, `max_event_depth_acquisition`, `pre-set_on_data_acquisition`, `offset_on_data_acquisition`
- Preprocessing/QC settings: `cube_voxel_side_length`, `FIX_GLITCHES_WITH_INTERPOLATION`, `minSnr`, `minEventDuration`, `maxEventDuration`, `minDepth`, `maxDepth`, `low_frequency`, `high_frequency`
- Geometry section: `STUDY_AREA`

When adding keys, keep the change backward-compatible where possible because all parent projects may read this file through the same submodule.

1.11 Development notes

- Use the `phd_conda_env_p10` interpreter/environment for this project.
- Keep this repository importable as a plain Python package directory.
- Do not write parent-project-specific code here unless it is genuinely shared.
- After changing `config.py`, `utils.py`, or `config.json`, update each parent repository's submodule pointer.
- To generate a PDF copy of this README:

```
pandoc README.md -o README.pdf \
--pdf-engine=xelatex \
--toc --toc-depth=3 \
--number-sections \
--highlight-style=tango \
-V geometry:margin=0.8in \
-V mainfont="DejaVu Serif" \
-V sansfont="DejaVu Sans" \
-V monofont="DejaVu Sans Mono" \
```

```
-V colorlinks=true \  
-V linkcolor=blue \  
-V urlcolor=blue
```